

Euron 10주차 발표 요약 — 토큰화 & 임베딩

자연어 처리(NLP)란?

텍스트를 숫자 벡터로 변환해야만 컴퓨터가 처리할 수 있으며, 그 첫 단계가 토큰화다.

인간 언어의 3가지 특성:

- **모호성**: 맥락에 따라 여러 의미
- **가변성**: 사투리, 신조어, 문체 등 다양한 형태
- **구조성**: 문법과 규칙에 따른 의미 해석 필요

토큰화 핵심 개념

용어	정의
Corpus (말뭉치)	분석 대상 텍스트 데이터셋 (뉴스·리뷰 등)
Token (토큰)	토큰화 결과물인 개별 단위 (단어·글자·형태소 등)
Tokenization	텍스트를 토큰으로 분리하는 과정 — NLP 첫 번째 단계
Tokenizer	토큰화를 수행하는 규칙/모델 — 방식에 따라 결과가 크게 달라짐

OOV 문제: 학습 어휘에 없는 단어 입력 시 발생 → 어휘 증가 = Sparse data & 차원의 저주

토큰화 방식 비교

방식	기준	장점	단점/한계
단어 토큰화	띄어쓰기	간단·직관적	조사·오타·부호에 취약, OOV↑
글자 토큰화	글자 단위	작은 사전, OOV↓	토큰에 의미 없음, 시퀀스 길어짐
자모 토큰화	초성/중성/종성	한국어 발음 패턴 처리	시퀀스 매우 길어짐(101자+)
형태소 토큰화	문법·구조	한국어 최적, 문맥 이해	신조어·외래어·오타자 취약

형태소 분석 라이브러리

라이브러리	특징
KoNLPy Okt	SNS 기반, 19개 품사, Java(JDK) 필요

KoNLPy 꼬꼬마	세종 말뭉치, 56개 품사 → 세밀하지만 느림
NLTK	영어 교육·연구용, Punkt + Perceptron Tagger (35종 품사)
spaCy	Cython 기반 머신러닝, 빠름·정확, 24개 이상 언어

하위 단어 토큰화(Subword Tokenization)

기존 형태소 분석기 한계: 신조어·오타자·고유어 처리 취약 → 어휘 사전 비대화

해결책: 단어를 빈번한 하위 단어 조각으로 분해 → OOV 완화 & 속도 향상

예) Reinforcement → Rein + force + ment

알고리즘	병합 기준	구현 라이브러리
바이트 페어 인코딩 (BPE)	빈도 기반 — 가장 자주 등장하는 글자 쌍 병합 반복	Sentencepiece (Google)
워드피스 (WordPiece)	확률 기반 — $\text{score} = f(x,y)/f(x) \cdot f(y)$ 수식으로 판단	Tokenizers (HuggingFace)

임베딩(Embedding)

토큰화 이후, 텍스트를 숫자 벡터로 변환(텍스트 벡터화)하는 과정이 필요하다.

방법	설명	한계
원-핫 인코딩	해당 단어 위치만 1, 나머지 0	희소성 큼, 의미 미반영
빈도 벡터화(BoW)	단어 등장 횟수를 벡터로	순서·문맥 무시
TF-IDF	단어 빈도 × 역문서 빈도로 중요도 가중	순서·문맥 무시
워드 임베딩 (Word2Vec)	유사 단어 → 유사 벡터, 분포 가설 기반	다의어·문맥 구분 어려움
동적 임베딩 (GPT·BERT)	문맥에 따라 다른 벡터 생성, 신경망 기반	대규모 리소스 필요

언어 모델 & N-gram

유형	설명
자기회귀 LM	이전 모든 토큰 참고 → 다음 단어 순서대로 예측 (출력이 다시 입력)
통계적 LM (마르코프)	단어 순서·빈도 기반 확률 계산. 긴 문맥 처리 한계, 희귀 표현 예측 어려움

N-gram	N개 단어 묶음으로 확률 계산. N=1유니그램, N=2바이그램, N=3트라이그램
GPT	인과적 LM — 단방향, 앞 단어로 다음 단어 예측
BERT	마스킹 LM — 양방향, 빈칸 단어를 문맥 전체로 예측

TF-IDF 핵심 정리

요소	수식	의미
TF	$\text{count}(t, d)$	문서 내 단어 등장 횟수
DF	$\text{count}(t \in d : d \in D)$	단어가 등장한 문서 수
IDF	$\log(D / (1 + DF))$	희귀 단어일수록 높음 (+1로 0방지, log로 과대값 방지)
TF-IDF	$TF \times IDF$	특정 문서에선 자주, 전체에선 희귀한 단어에 높은 가중치

한계: 문장 순서·문맥 미반영 / 벡터가 단어 의미를 담지 않음 → 사이킷런 TfidfVectorizer
 사용: `fit()` → `transform()` (또는 `fit_transform()`)

과정: 사람 → 텍스트 입력 → 토큰화 → 수치화(임베딩) → 모델 학습